

MGCL: Multiorder Graph Neural Network With Cross-Learning for Multivariate Time-Series Anomaly Detection

Zhe Cui^{ID}, Di Zang^{ID}, Junqi Zhang^{ID}, *Senior Member, IEEE*, and Keshuang Tang^{ID}

Abstract—Multivariate time-series (MTS) anomaly detection is critical for industrial systems. However, for inter-variable dimensional modeling, existing work mainly focuses on low-order dependencies between variable pairs, overlooking high-order interactions among multiple variables. Additionally, for temporal dimension modeling, time series are influenced by various system factors, posing challenges in simultaneously capturing long-term and short-term dependencies. To address these challenges, we propose a multiorder graph neural network with cross-learning (MGCL). First, we design a multiscale temporal module that captures long-term seasonal dependencies and short-term trend dependencies by employing a temporal global attention network (TGAN) and a gated temporal convolution network (GTCN), respectively. Second, we construct an adaptive hypergraph generator to explicitly represent the dynamic high-order interactions among multiple variables. Finally, we develop a multiorder graph cross-learning module, which integrates a dynamic memory graph convolution network (DMGCN), a tri-directional diffusion hypergraph convolution network (3D-HGCN), and a cross-learner to comprehensively capture multiorder inter-variable dependencies. Extensive experiments on eight real-world datasets demonstrate that MGCL outperforms state-of-the-art baselines.

Index Terms—Anomaly detection, cross-learning, graph neural network (GNN), hypergraph, multivariate time series (MTS).

I. INTRODUCTION

MULTIVARIATE time series (MTS) are prevalent in the Internet of Things (IoT), industry, transportation, and power grids. Anomaly detection in MTS aims to identify behaviors that deviate from expected patterns. However, due to the high complexity and susceptibility of MTS, manual monitoring is no longer feasible. Therefore, automated MTS

anomaly detection has become indispensable for fault diagnosis and emergency response [1], [2].

Early MTS anomaly detection methods primarily rely on machine learning techniques [3]. However, these approaches often encounter the “curse of dimensionality” when dealing with high-dimensional MTS. Specifically, as the dimensionality of time series increases, model performance tends to degrade. In contrast, deep learning methods have gained significant attention for automatically capturing nonlinear features in high-dimensional data, including supervised, semi-supervised, and unsupervised methods. Supervised methods rely on predefined labels and perform well when detecting known anomaly patterns. However, due to the diversity of anomaly classes, high-quality labels often require extensive manual annotation. Furthermore, the extreme imbalance between positive and negative samples in anomaly data limits the model’s generalization ability. Although semi-supervised methods improve generalization by learning from limited labeled data, they still struggle to recognize diverse anomaly patterns due to the limited label categories. Unsupervised methods bypass the need for labels, effectively addressing challenges related to label scarcity and diverse anomaly patterns. Early unsupervised approaches, such as RNNs [4], GRUs [5], and LSTMs [6], excel at capturing temporal dependencies but overlook the strong correlations between different time series, limiting their ability to model inter-variable dependencies. Subsequently, unsupervised methods based on graph neural networks (GNNs) [7] and Transformer [8] are introduced into MTS anomaly detection, demonstrating significant potential in modeling inter-variable relationships. Nevertheless, they still face the following limitations.

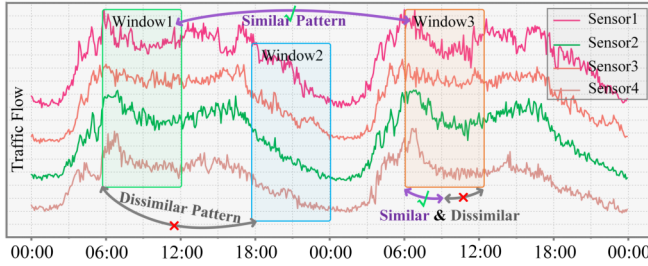
First, in terms of temporal dimension modeling, time series often exhibit complex multiscale characteristics. Fig. 1(a) shows two days of traffic flow data recorded by different sensors. Specifically, the traffic system displays dissimilar patterns during peak and off-peak periods (Window 1 and Window 2). Additionally, peak periods on different days often exhibit similar patterns (Window 1 and Window 3). This indicates strong long-term seasonal dependencies inherent in the time series. Although models like RNNs have advantages in capturing long-term dependencies, they are often limited by the size of the time window and typically suffer from gradient explosion when processing long sequences. Moreover, the time series also contains more complex short-term patterns. For

Received 8 January 2025; revised 9 May 2025; accepted 23 May 2025. Date of publication 9 June 2025; date of current version 18 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476200 and Grant 62472313, in part by the 2024 Shanghai “Science and Technology Innovation Action Plan” International Scientific and Technological Cooperation Program under Grant 24510714400, and in part by the Innovation Program of Shanghai Municipal Education Commission under Grant 202101070007E00098. The Associate Editor coordinating the review process was Dr. Mahnoosh Tajmiri. (Corresponding authors: Di Zang; Junqi Zhang.)

Zhe Cui, Di Zang, and Junqi Zhang are with the College of Computer Science and Technology and the Key Laboratory of Embedded System and Service Computing, Ministry of Education, Tongji University, Shanghai 200092, China (e-mail: cuizhe117@tongji.edu.cn; zangdi@tongji.edu.cn; zhangjunqi@tongji.edu.cn).

Keshuang Tang is with the Key Laboratory of Road and Traffic Engineering, Ministry of Education, and the College of Transportation Engineering, Tongji University, Shanghai 200092, China.

Digital Object Identifier 10.1109/TIM.2025.3577826



(a) The multi-scale characteristics in temporal dimension modeling.



(b) The dynamic and multi-order characteristics in inter-variable dimension modeling.

Fig. 1. Intricacy of temporal and inter-variable dependencies modeling in a system. (a) In temporal dimension modeling, similar and dissimilar patterns represent consistent or divergent data variation trends, respectively. These patterns across time windows reveal the multiscale temporal characteristics of traffic flow, encompassing short-term and long-term trends, and reflect the hierarchical complexity of time series at various temporal granularities. (b) In inter-variable dimension modeling, similar patterns indicate consistent trends between sensors or regions composed of multiple sensors, highlighting strong correlations or synergistic effects. Conversely, dissimilar patterns reflect local disparities or independent characteristics. Thus, dynamic low-order interactions between sensors manifest as pairwise dependencies between topological nodes, whereas dynamic high-order interactions across regions reveal complex associations among multiple topological nodes.

instance, there are both similar and dissimilar patterns in Window 3, reflecting the complexity and diversity of short-term trend dependencies. Therefore, to comprehensively understand the complex characteristics of time series, it is essential to capture both long-term and short-term dependencies.

Second, in terms of inter-variable dimensional modeling, the relationships among variables often exhibit dynamics and multiorder characteristics. Fig. 1(b) illustrates an example of traffic conditions during peak hours on a workday. During peak hours, traffic flow usually travels between residential and business areas, with variable relationships evolving continuously over time. This highlights the dynamic correlations among variables. However, most existing MTS anomaly detection methods rely on static graph structures, lacking the ability to dynamically model inter-variable dependencies. Furthermore, both low-order and high-order interactions coexist among variables, reflecting their multiorder characteristics. For example, as shown in Fig. 1(b), low-order interactions may occur between topological node pairs within a business area, while high-order interactions arise from regional disparities. Traffic patterns between business areas often display similarities, while those between leisure and residential areas may be vastly dissimilar. These regional disparities underscore the complexity and significance of high-order interactions among multiple topological nodes. However, existing methods, such as GNNs

and Transformer, primarily model inter-variable dependencies based on non-Euclidean graph structures, neglecting dynamic evolution and high-order interactions among variables.

To address the first limitation, we propose a multiscale temporal module. It utilizes fast Fourier transform (FFT) to decompose raw time series into seasonal and trend components. Then, the temporal global attention network (TGAN) and gated temporal convolution network (GTCN) are employed to capture long-term seasonal and short-term trend dependencies, respectively. This enhances the model's ability to perceive multiscale temporal features.

To overcome the second limitation, we propose an adaptive hypergraph generator. It constructs dynamic hypergraphs based on the spatial-temporal characteristics of input MTS, explicitly representing complex high-order interactions and their evolution among variables. Furthermore, we design a multiorder graph cross-learning module, which captures low-order dependencies via the dynamic memory graph convolutional network (DMGCN) and high-order dependencies via the tri-directional diffusion hypergraph convolution network (3D-HGCN). Additionally, a cross-learner further facilitates complementary learning and deep fusion of multiorder dependencies.

In the end, the model proposed for the above limitations is named multiorder GNN with cross-learning, namely MGCL. To the best of our knowledge, this is the first work to apply dynamic multiorder graphs for MTS anomaly detection. The main contributions of this article are summarized as follows.

- 1) We propose an MGCL framework for MTS anomaly detection, which effectively captures multiscale temporal dependencies and models multiorder inter-variable dependencies.
- 2) We design a multiscale temporal module that captures long-term seasonal and short-term trend dependencies, thereby thoroughly perceiving multiscale temporal dependencies.
- 3) We construct an adaptive hypergraph generator that automatically builds hypergraphs based on the spatial-temporal characteristics of MTS, explicitly representing high-order interactions and their evolution among variables. Meanwhile, we design a multiorder graph cross-learning module aimed at capturing both low-order and high-order inter-variable dependencies, promoting their complementary learning and deep fusion.
- 4) We conduct extensive experiments on eight real-world datasets. The results demonstrate that MGCL achieves state-of-the-art performance and validates its effectiveness in anomaly localization and diagnosis.

The remainder of this article is organized as follows. Section II summarizes the related work. Section III presents preliminary. Section IV introduces our method. Section V shows experiment results. Finally, Section VI concludes this article.

II. RELATED WORK

A. Traditional Methods for Anomaly Detection

Early anomaly detection studies primarily rely on statistical theories, such as ARIMA [9] and PCA [10]. These methods

assume that time series follows a linear structure, which prevents them from capturing nonlinear features. Subsequently, machine learning methods, such as One-Class SVM [11] and STS-AD [12], improve detection performance by capturing nonlinear features. However, these methods rely on manually designed features and suffer significant performance degradation when dealing with high-dimensional data.

B. GNNs for Anomaly Detection

Deep unsupervised methods are widely adopted for handling high-dimensional, hard-to-label anomalous data. RNN-based approaches [13] excel at capturing temporal dependencies within individual time series but face gradient explosion issues with long sequences and cannot model inter-variable dependencies. Although CNN-based methods [14] can capture inter-variable dependencies, they assume MTS follow grid-like, Euclidean structures. However, in real-world IoT, power, and transportation systems, sensor distributions exhibit unique non-Euclidean structures, motivating the use of GNN [7].

Early GNN methods primarily rely on static graphs to model inter-variable dependencies. GDN [15] represents inter-variable relationships as non-Euclidean graph structures and detects anomalies based on prediction errors. MSCRED [16] employs a graph feature matrix to encode inter-variable relationships and integrates LSTM to capture temporal dependencies, detecting anomalies via reconstruction errors. MTAD-GAT [17] utilizes a graph attention network to jointly learn temporal and inter-variable dependencies. However, these methods assume static graph structures, neglecting the dynamic evolution of variable relationships.

To overcome the limitations of static graphs, recent methods have explored dynamic graphs. MAGNN [18] proposes a multiscale adaptive GNN that dynamically models inter-variable dependencies through multiscale temporal decomposition and adaptive graph learning. However, its multiscale modeling fails to explicitly distinguish between global and local temporal dependencies and overlooks the multiorder characteristics of variables. Additionally, AddGraph [19] combines dynamic graphs with GRU to capture anomalous patterns. GReLeN [20] models inter-variable dependencies using dynamic graphs and a random graph relation learning strategy. MEGA [1] integrates discrete wavelet transforms with graph autoencoders (AEs) to enhance anomaly detection. CrossFuN [2] further improves detection by combining dynamic graphs with time-frequency domain information.

C. Transformer for Anomaly Detection

Benefiting from advancements in self-attention mechanism, Transformer has demonstrated exceptional performance. TranAD [21] utilizes a self-attention-based sequence encoder to capture temporal dependencies and infer anomaly patterns. Anomaly Transformer [22] improves the conventional self-attention mechanism by introducing an anomaly attention mechanism, achieving effective detection by maximizing the distinction between normal and abnormal patterns. DTAAD [23] integrates TCN with Transformer, enabling it to capture both temporal and inter-variable dependencies. However, in

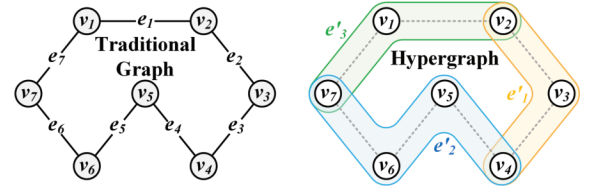


Fig. 2. Traditional graph and hypergraph structure.

modeling temporal dependencies, these methods fail to fully consider the multiscale temporal characteristics. Moreover, in modeling inter-variable dependencies, they overlook higher-order interactions among multiple variables.

In summary, compared to prior work, our proposed MGCL offers the following advantages.

- 1) MGCL employs FFT for multiscale decomposition of raw time series and constructs TGAN and GTCN to effectively capture long-term seasonal and short-term trend dependencies.
- 2) MGCL adaptively constructs hypergraphs based on the spatial-temporal characteristics of MTS, explicitly representing high-order interactions and their dynamic evolution among variables. This overcomes the limitations of traditional Euclidean graph structures, making it more suitable for real-world data.
- 3) MGCL leverages DMGCN and 3D-HGCN to model low-order and high-order inter-variable dependencies, respectively, and employs a cross-learner to promote their complementary learning and deep fusion.

III. PRELIMINARY

A. Definitions

1) *Traditional Graph Structure*: It is $G = (V; E; \mathcal{A})$. It consists of a node (variable) set $V = \{v_1, v_2, \dots, v_N\}$ ($|V| = N$), and an edge set $E = \{e_1, e_2, \dots, e_M\}$ ($|E| = M$) that connects paired nodes. Edges in E represent low-order pairwise relationships between variables, as shown in Fig. 2. An adjacency matrix $\mathcal{A}$ defines these relationships: when variable v_i is correlated with v_j , $\mathcal{A}(v_i, v_j) = 1$. Otherwise, $\mathcal{A}(v_i, v_j) = 0$. Notably, $\mathcal{A}$ may not always be available in all systems. In such cases, we adopt an adaptive embedding matrix $\mathcal{Z}$ as a substitute.

2) *Hypergraph Structure*: It is defined as $\mathcal{G} = (V; \mathcal{E}; \mathcal{H})$, an extension of a traditional graph. It includes a hyperedge set $\mathcal{E} = \{e'_1, e'_2, \dots, e'_M\}$ ($|\mathcal{E}| = M$) that connects multiple nodes, as shown in Fig. 2. Hyperedges represent high-order inter-variable interactions. An incidence matrix $\mathcal{H}$ encodes the relationships between nodes and hyperedges: if a node v belongs to a hyperedge e' , then $\mathcal{H}(v, e') = 1$. Otherwise, $\mathcal{H}(v, e') = 0$.

B. Problem Formalization

A system is represented as G . MTS is $X \in \mathbb{R}^{N \times T}$, where N represents the number of variables and T is the length of the time series. At time step t , MTS is simplified as X_t . The objective of anomaly detection is to learn a function S from historical h time steps of data without requiring known labels,

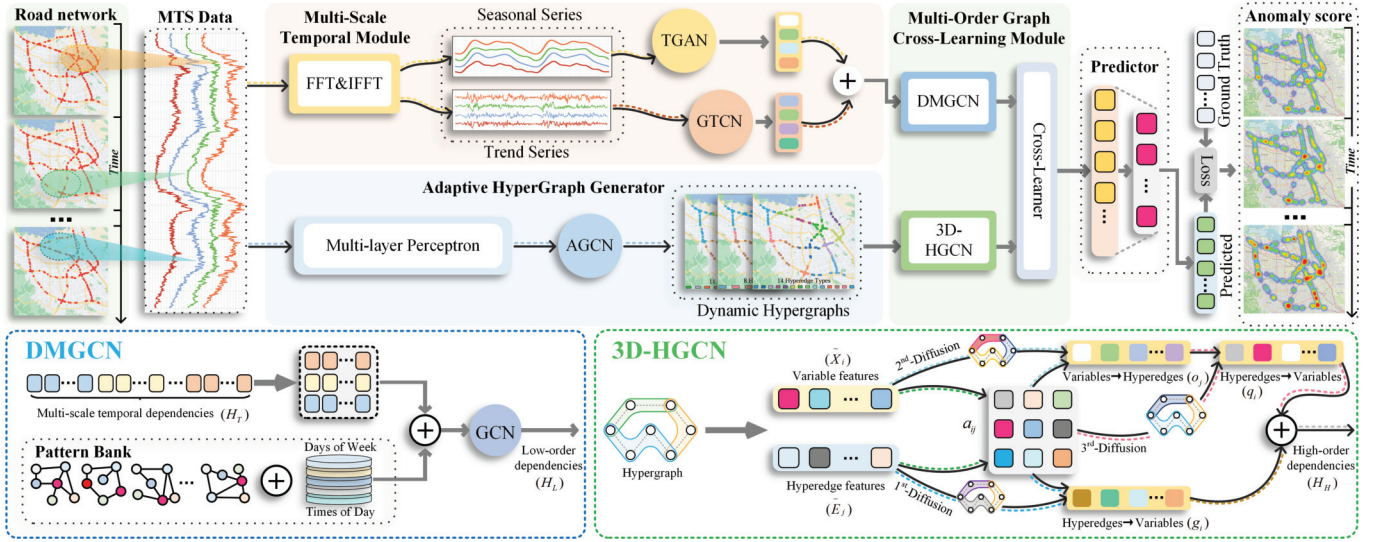


Fig. 3. Architecture of the proposed MGCL consists of four main modules: a multiscale temporal module, an adaptive hypergraph generator, a multiorder graph cross-learning module, and a predictor. The multiscale temporal module leverages TGAN and GTCN to capture long-term seasonal and short-term trend dependencies within the time series. The adaptive hypergraph generator dynamically constructs hypergraphs based on the spatial-temporal characteristics of input MTS, explicitly representing high-order inter-variable interactions and their dynamic evolution. The multiorder graph cross-learning module integrates DMGCN, 3D-HGCN, and a cross-learner, which captures low-order dependencies in traditional graphs and high-order dependencies in hypergraphs, while promoting their complementary learning and deep fusion. The predictor refines feature dimensions and produces the final prediction results.

to detect the anomaly state $\mathcal{D} \in \{0, 1\}$ (1 represents anomaly) within a future time window W

$$[X_{(t-h):t}, G] \xrightarrow{\mathcal{S}} \mathcal{D}_{t:(t+W)}. \quad (1)$$

IV. METHODOLOGY

A. Framework Overview

The structure of MGCL is shown in Fig. 3. First, the MTS is processed in parallel by the multiscale temporal module and the adaptive hypergraph generator, which, respectively, capture multiscale temporal dependencies and construct dynamic hypergraphs. Second, the multiorder graph cross-learning module comprehensively models multiorder inter-variable dependencies. Finally, the predictor refines the channel dimensions of multiorder dependencies and generates prediction results. Detailed descriptions of each module are provided below.

B. Multiscale Temporal Module

In real-world systems, time series exhibit complex characteristics, with both long-term seasonal and short-term trend dependencies. However, models are often constrained by the size of a time window. When a window is large, the model tends to overlook short-term detailed features. Conversely, when a window is small, the model struggles to capture long-term dependencies and is prone to gradient explosion. To address these challenges, we propose a multiscale temporal module, comprising three components.

1) *Temporal Scale Decomposition*: This process first applies FFT to convert the original MTS into multiple frequency signals

$$X_{\text{FFT}} = \text{FFT}(X) \quad (2)$$

$$f_1, \dots, f_k = \arg\text{Top}K(\text{Amp}(X_{\text{FFT}})) \quad (3)$$

$$X_{\text{FFT}}^{\text{main}} = \{X_{\text{FFT}}[f_1], \dots, X_{\text{FFT}}[f_k]\} \quad (4)$$

where X represents MTS data, X_{FFT} is a frequency domain signal after FFT processing, $\text{Amp}(\cdot)$ denotes an amplitude function, $\arg\text{Top}K(\cdot)$ selects the top K frequency components based on their amplitudes, and $X_{\text{FFT}}^{\text{main}}$ represents a set of multifrequency signals composed of the top K amplitude components.

Subsequently, a seasonal time series X_{Season} , which represents long-term variations in time series, is extracted using inverse FFT (IFFT) based on $X_{\text{FFT}}^{\text{main}}$. Then, a trend time series X_{Trend} , representing short-term variations in time series, is obtained through a series reduction process. This process is expressed as follows:

$$X_{\text{Season}} = \text{IFFT}(X_{\text{FFT}}^{\text{main}}) \quad (5)$$

$$X_{\text{Trend}} = X - X_{\text{Season}}. \quad (6)$$

2) *Global Temporal Attention Convolution*: Seasonal series typically exhibit smooth variations, requiring the capture of dependencies on a global scale. Therefore, we construct the TGAN

$$a_{ti} = \frac{\exp((\mathcal{W}^Q X_{\text{Season},t})^T (\mathcal{W}^K X_{\text{Season},i}))}{\sum_{m=1}^t \exp((\mathcal{W}^Q X_{\text{Season},t})^T (\mathcal{W}^K X_{\text{Season},m}))} \quad (7)$$

$$H_{TL} = \text{TGAN}(X_{\text{Season}}) = \sum_{i=1}^t a_{ti} (\mathcal{W}^V X_{\text{Season},i}) \quad (8)$$

where $\exp(\cdot)$ denotes exponential function. $\mathcal{W}^Q$, $\mathcal{W}^K$, and $\mathcal{W}^V$ are trainable parameters. a_{ti} represents the correlation between time step t and time step i . H_T indicates long-term seasonal dependencies captured by GTAN.

3) *Gated Temporal Convolution*: Trend series often exhibit complex local fluctuations. To effectively capture their short-term trend dependencies, we construct the GTCN

$$\text{TCN}(X_{\text{Trend}}) = \text{Conv}_s^{\text{dil}}(X_{\text{Trend}}) \quad (9)$$

$$\begin{aligned} \mathbf{H}_{\text{TS}} &= \text{GTCN}(X_{\text{Trend}}) = \text{Tanh}(\text{TCN}(X_{\text{Trend}})) \\ &\odot \text{Sigmoid}(\text{TCN}(X_{\text{Trend}})) \end{aligned} \quad (10)$$

where $\text{Conv}_s^{\text{dil}}(\cdot)$ represents a temporal convolution operation with s kernel size and dil dilation rate. $\text{Tanh}(\cdot)$ and $\text{Sigmoid}(\cdot)$ are Tanh and Sigmoid activation functions, respectively. $\odot$ indicates Hadamard product. $\mathbf{H}_{\text{TS}}$ is short-term trend dependencies captured by GTCN.

Finally, the short-term trend dependencies $\mathbf{H}_{\text{TS}}$ are combined with the long-term seasonal dependencies $\mathbf{H}_{\text{TL}}$ to comprehensively capture multiscale temporal dependencies

$$\mathbf{H}_T = \mathbf{H}_{\text{TS}} + \mathbf{H}_{\text{TL}}. \quad (11)$$

C. Adaptive HyperGraph Generator

In real-world systems, sensors of the same type exhibit homogeneity, whereas sensors of different types display heterogeneity. While low-order inter-variable interactions are important, high-order interactions among multiple variables cannot be overlooked. Although prior studies have utilized traditional hypergraphs [24] to represent such higher-order inter-variable interactions, several limitations remain. 1) Traditional hypergraphs are constructed using K -means clustering to define hyperedges, requiring manual tuning of the K value. However, real-world systems are diverse, making it challenging to determine an appropriate K value, which may also vary with system changes. 2) Traditional hypergraph construction is applied directly to the complete training set, resulting in a static and unchangeable structure. Thus, it lacks flexibility and the ability to adjust in real time. To address these issues, we design an adaptive hypergraph generator. The generator not only dynamically constructs hypergraphs but also adjusts their structure in real time, explicitly modeling the impact of anomalies on a system. For example, it can describe fault propagation in a power grid or congestion diffusion in a traffic system.

1) *Feature Dimension Expansion*: The adaptive hypergraph generator first employs a standard multilayer perceptron (MLP) to adjust the channels of the original MTS to facilitate subsequent feature exploration. The process is expressed as

$$\mathbf{H}' = \text{MLP}(\mathbf{X}) \quad (12)$$

where $\mathbf{X}$ represents the original MTS data, and $\mathbf{H}'$ denotes the hidden features processed by the MLP.

2) *Adaptive Graph Convolution*: To cluster similar variables into a hyperedge, we construct an adaptive graph convolutional network (AGCN) to capture inter-variable relationships in $\mathbf{H}'$. The adaptive adjacency matrix is

$$\mathbf{A}_a = \text{Soft}(\text{ReLU}(\mathbf{Z}_1 \mathbf{Z}_2^T)) \quad (13)$$

where $\mathbf{Z}_1$ and $\mathbf{Z}_2$ are adaptive embedding matrices. $\text{Soft}(\cdot)$ and $\text{ReLU}(\cdot)$ are Softmax and ReLU functions, respectively. In real-world systems, inter-variable interactions are often

directional, such as traffic flows in transportation systems or communication links in IoT systems. Based on the predefined adjacency matrix $\mathbf{A}$, we construct a forward propagation matrix $\mathbf{D}_{\mathcal{F}}$ and a backward propagation matrix $\mathbf{D}_{\mathcal{B}}$

$$\mathbf{D}_{\mathcal{F}} = \mathbf{A} / \text{rowsum}(\mathbf{A}), \quad \mathbf{D}_{\mathcal{B}} = \mathbf{A}^T / \text{rowsum}(\mathbf{A}^T). \quad (14)$$

Furthermore, we design the AGCN with a dual-propagation mechanism, formulated as

$$\begin{aligned} \mathbf{H}_A &= \text{AGCN}(\mathbf{H}') = \sum_{k=0}^K (\mathbf{D}_{\mathcal{F}}^k \mathbf{H}' \mathbf{W}_1 + \mathbf{D}_{\mathcal{B}}^k \mathbf{H}' \mathbf{W}_2) \\ &+ \mathbf{A}_a \mathbf{H}' \mathbf{W}_3 \end{aligned} \quad (15)$$

where $\mathbf{W}_1$, $\mathbf{W}_2$, and $\mathbf{W}_3$ are trainable parameters. k indicates a diffusion step length. $\mathbf{H}_A$ is inter-variable relationships captured by AGCN.

3) *Hypergraph Construction*: Based on $\mathbf{H}_A$, we cluster highly similar variables into a hyperedge and construct the hypergraph incidence matrix $\mathbf{H}$ to represent high-order inter-variable relationships

$$\mathbf{H} = \text{Max}(\text{Soft}(\mathbf{H}_A) \mathbf{W}_4) \quad (16)$$

where $\mathbf{W}_4$ is a trainable parameter. $\text{Max}(\cdot)$ selects the maximum value of each row in the matrix, indicating the variable with the highest similar score. Based on the incidence matrix $\mathbf{H}$, we construct the hypergraph $\mathcal{G} = (\mathbf{V}; \mathcal{E}; \mathbf{H})$ to support the subsequent modeling of high-order inter-variable dependencies.

4) *Convergence of Hypergraph*: The adaptive hypergraph generator dynamically constructs a hypergraph $\mathcal{G}$ from MTS data $\mathbf{X}$, explicitly representing high-order interactions and their dynamic evolution among variables. To address convergence challenges arising from dynamic adjustments, we ensure stability through mathematical constraints and optimized strategies.

Specifically, a global optimization constraint L , as defined in (38), is imposed during training. This constraint ensures convergence toward the global optimum, mitigating random drift of the hypergraph. Concurrently, the Adam optimizer updates the model parameters as follows:

$$\mathcal{P}_{n+1} = \mathcal{P}_n - \mu \nabla_{\mathcal{P}} L \quad (17)$$

where $\mathcal{P}_n$ represents the model parameters at iteration n , μ is the learning rate, and $\nabla_{\mathcal{P}} L$ denotes the gradient of the loss L . Since $\mathbf{H}$ is driven by $\mathbf{X}$ and the prediction results depend on $\mathbf{H}$, optimizing L indirectly constrains the hypergraph adjustment process. The Adam optimizer adaptively adjusts the learning rate by computing the first- and second-order moment estimates of the gradients, thereby preventing divergence.

Additionally, to mitigate significant fluctuations of the hypergraph, we introduce a smoothness regularization term

$$L_{\text{smooth}} = \eta \sum_{t=2}^T \|\mathbf{H}^{(t)} - \mathbf{H}^{(t-1)}\|_F^2 \quad (18)$$

where η is the regularization coefficient, and $\|\cdot\|_F$ denotes the Frobenius norm. This smoothness constraint ensures numerical stability by penalizing abrupt changes in the hypergraph

between consecutive time steps. Moreover, from a theoretical perspective, we analyze the convergence of the hypergraph and use (18) to derive its structural variation bounds

$$\|\mathcal{H}^{(t)} - \mathcal{H}^{(t-1)}\|_F \leq \sqrt{L_{\text{smooth}}/\eta(T-1)}. \quad (19)$$

The bound ensures that the hypergraph's variation is tightly constrained by regularization, preventing divergence. In summary, these mathematical constraints and optimization strategies promote the stable convergence of the hypergraph generator during training.

D. Multiorder Graph Cross-Learning Module

To capture multiorder inter-variable dependencies, we design the multiorder graph cross-learning module comprising three submodules: DMGCN, 3D-HGCN, and a cross-learner.

1) *Dynamic Memory Graph Convolution*: Although prior methods using GCNs [25] have proven effective in capturing inter-variable dependencies, limitations still remain. 1) Inter-variable relationships in a graph structure may change over time. However, traditional graph structures are static, meaning their adjacency matrix $\mathcal{A}$ is time-invariant and cannot represent the dynamic inter-variable associations. 2) Due to the sparsity of anomalous data, MTS anomaly detection methods need to learn normal patterns. The periodic characteristics of historical data are important for learning these patterns. For instance, traffic flow often exhibits similar patterns on the same weekdays (days of week), and electricity consumption often follows similar behavior during the same times of the day (times of day). However, current methods overlook these historical periodic patterns. Therefore, we construct a dynamic adjacency matrix $\mathcal{A}_d$ to represent the time-varying relationships among variables. Additionally, to enhance the model's memory of periodic patterns, we introduce a historical pattern bank for each variable and construct a pattern adjacency matrix $\mathcal{A}_p$. They are expressed as

$$\mathcal{A}_d = \frac{\exp(\mathbf{H}_T^T(\dots, i) \mathbf{H}_T(\dots, j))}{\sum_{i=1}^N \exp(\mathbf{H}_T^T(\dots, i) \mathbf{H}_T(\dots, j))} \quad (20)$$

$$\mathcal{A}_p = \frac{\exp(\mathbf{H}_T^T(\dots, i) \mathbf{B}(\dots, j))}{\sum_{i=1}^N \exp(\mathbf{H}_T^T(\dots, i) \mathbf{B}(\dots, j))} \quad (21)$$

where $\mathbf{H}_T^T$ represents the transpose of the multiscale temporal features $\mathbf{H}_T$. N is the number of variables. $\mathbf{B}$ indicates the pattern bank, which stores historical periodic features (times of day and days of week) for each variable to assist in modeling normal patterns.

Furthermore, we fuse the two adjacency matrices to construct DMGCN

$$\mathcal{Q} = \text{FC}(\mathcal{A}_d \parallel \mathcal{A}_p) \quad (22)$$

$$\mathbf{H}_L = \text{DMGCN}(\mathcal{Q}, \mathbf{H}_T \mathbf{W}_5) \quad (23)$$

where $\parallel$ denotes a feature concatenation operation. $\text{FC}(\cdot)$ represents a fully connected layer. $\mathcal{Q}$ is the fused dynamic memory adjacency matrix. $\mathbf{W}_5$ indicates a trainable weight matrix. $\mathbf{H}_L$ represents low-order inter-variable dependencies captured by DMGCN.

2) *Tri-Directional Diffusion Hypergraph Convolution*: To effectively capture high-order dependencies of dynamic hypergraphs, we need to construct a Hypergraph Convolutional Network (HGCN). Previous HGCNs [26] and their advanced variants [27] rely solely on variable features, aggregating variable features into hyperedges and subsequently updating variable features based on hyperedge features. However, these methods focus only on local features of hypergraphs while neglecting their global features. Therefore, we propose the 3D-HGCN, which comprehensively captures high-order inter-variable dependencies from both global and local perspectives.

First, through a dimensional transformation layer, variables and hyperedges in a dynamic hypergraph $\mathcal{H}$ are transformed into feature representations to facilitate subsequent tri-directional feature diffusion

$$\tilde{\mathbf{X}}_i = \mathbf{W}_6 \mathbf{X}_i + \mathbf{b}_1 \quad (24)$$

$$\tilde{\mathbf{E}}_j = \mathbf{W}_7 \mathbf{E}_j + \mathbf{b}_2 \quad (25)$$

where $\mathbf{X}_i$ is the time series of variable v_i . $\tilde{\mathbf{X}}_i$ represents the feature representation of variable v_i after the dimensional transformation. Similarly, $\mathbf{E}_j$ is the embedding of hyperedge e'_j . $\tilde{\mathbf{E}}_j$ denotes the feature representation of hyperedge e'_j after the dimensional transformation. $\mathbf{W}_6$, $\mathbf{W}_7$, $\mathbf{b}_1$, and $\mathbf{b}_2$ are trainable parameters.

Next, based on the feature representations of variables and hyperedges, the interaction similarity between variables and hyperedges is computed using a self-attention mechanism

$$a_{ij} = \frac{\exp(\tilde{\mathbf{X}}_i \tilde{\mathbf{E}}_j^T)}{\sum_{e_r \in v_i} \exp(\tilde{\mathbf{X}}_i \tilde{\mathbf{E}}_r^T)} \quad (26)$$

where $\tilde{\mathbf{E}}_j^T$ is the transpose of hyperedge feature representations $\tilde{\mathbf{E}}_j$. a_{ij} represents the similarity score between node v_i and hyperedge e'_j , which can dynamically change over time.

Then, 3D-HGCN performs a tri-directional diffusion process. Specifically, hyperedge features propagate to variables, variable features propagate to hyperedges, and hyperedge features aggregate to variables. These processes are expressed as follows:

$$\mathbf{g}_i = \frac{\sum_{e_j \in e_v} a_{ij} \tilde{\mathbf{E}}_j}{\sum_{e_j \in e_v} a_{ij}} \quad (27)$$

$$\mathbf{o}_j = \frac{\sum_{v_i \in e_j} a_{ij} \tilde{\mathbf{X}}_i}{\sum_{v_i \in e_j} a_{ij}} \quad (28)$$

$$\mathbf{q}_i = \frac{\sum_{e_j \in e_v} a_{ij} \mathbf{o}_j}{\sum_{e_j \in e_v} a_{ij}}. \quad (29)$$

Finally, a feature aggregation operation is performed to obtain high-order representations after tri-directional diffusion

$$\mathbf{H}_H = \text{Sigmoid}\left(\sum_{i=1}^N \mathbf{g}_i + \sum_{i=1}^N \mathbf{q}_i\right) \quad (30)$$

where N indicates the number of variables, and $\mathbf{H}_H$ represents high-order inter-variable dependencies captured by 3D-HGCN.

3) *Cross-Learner*: Low-order inter-variable dependencies $\mathbf{H}_L$ and high-order inter-variable dependencies $\mathbf{H}_H$ inevitably have overlapping and complementary features. To eliminate redundant features and promote complementary advantages, we design a cross-learner

$$A_{LH} = \frac{\exp(\mathbf{H}_H^T(\dots, i) \mathbf{H}_L(\dots, j))}{\sum_{i=1}^N \exp(\mathbf{H}_H^T(\dots, i) \mathbf{H}_L(\dots, j))} \quad (31)$$

$$A_{HL} = \frac{\exp(\mathbf{H}_L^T(\dots, i) \mathbf{H}_H(\dots, j))}{\sum_{i=1}^N \exp(\mathbf{H}_L^T(\dots, i) \mathbf{H}_H(\dots, j))} \quad (32)$$

where A_{LH} and A_{HL} represent the correlation scores between low→high and high→low dependencies, respectively.

Then, based on the correlation scores, cross-learning is performed as follows:

$$\mathbf{H}'_L = \omega \odot \mathbf{H}_L + \varpi \odot (A_{LH} \mathbf{H}_H) \quad (33)$$

$$\mathbf{H}'_H = \xi \odot \mathbf{H}_H + \gamma \odot (A_{HL} \mathbf{H}_L) \quad (34)$$

where ω , ϖ , ξ , and γ are trainable parameters. $\mathbf{H}'_L$ ($\mathbf{H}_L$) and $\mathbf{H}'_H$ ($\mathbf{H}_H$) denote the updated low-order and high-order dependencies at l ($l-1$) layer, respectively.

Finally, $\mathbf{H}'_L$ and $\mathbf{H}'_H$ are fused to achieve multiorder inter-variable dependencies modeling:

$$\mathbf{H}_M = \mathbf{H}'_L + \mathbf{H}'_H. \quad (35)$$

E. Predictor

The multiorder inter-variable dependencies $\mathbf{H}_M$ are fed into the predictor to generate predictions $\mathbf{Y}$, as shown in Fig. 3. This process is expressed as

$$\mathbf{Y} = \text{MLP}(\mathbf{H}_M) \quad (36)$$

where the MLP consists of two fully connected layers and functions as a non-autoregressive method to generate predictions, effectively avoiding the issue of error accumulation.

F. Loss Function

In MTS anomaly detection, the choice of the loss function has a significant impact on model performance [28]. To effectively handle anomaly detection in different time periods, we propose a segmented loss function that combines mean absolute error (MAE) and root mean squared error (RMSE). For time periods with abrupt changes (e.g., sensor anomalies), RMSE is employed to amplify the differences between predictions and actual values, ensuring MGCL accurately detects anomalies with large fluctuations. For stable periods with minor changes under normal conditions, MAE is used to avoid over-penalizing small errors, thereby maintaining accuracy during stable periods. The segmented loss function is formulated as

$$f(\hat{\mathbf{Y}}_t, \mathbf{Y}_t) = \begin{cases} |\hat{\mathbf{Y}}_t - \mathbf{Y}_t|, & \frac{Y_t - Y_{t-1}}{Y_t} < \psi \\ (\hat{\mathbf{Y}}_t - \mathbf{Y}_t)^2, & \text{otherwise} \end{cases} \quad (37)$$

$$L(\hat{\mathbf{Y}}_{(t+1):(t+T)}, \mathbf{Y}_{(t+1):(t+T)}) = \sum_{t=1}^T f(\hat{\mathbf{Y}}_t, \mathbf{Y}_t) \quad (38)$$

where $\mathbf{Y}_t$ represents a predicted value by MGCL, $\hat{\mathbf{Y}}_t$ is a ground truth, and ψ is a hyperparameter. The segmented loss function improves the overall performance of anomaly detection by enhancing MGCL's sensitivity to anomalies while maintaining a stable fitting effect during normal periods.

Algorithm 1 Training of MGCL

Input: X_{train} : trained MTS series
Output: $\mathcal{P}$: optimal parameters for MGCL

```

1 begin
2   for  $epoch$  do
3     for  $X'$  in  $X_{train}$  do
4       Getting  $X_{Season}$ ,  $X_{Trend}$  by  $FFT(X')$ 
5       Capturing multiscale temporal features  $\mathbf{H}_T$ 
6       Constructing hypergraph  $\mathcal{H}$  and dynamic
         memory graph  $\mathcal{Q}$ 
7       Modeling inter-variable dependencies  $\mathbf{H}_M$ 
8       Getting prediction results  $\mathbf{Y}$ 
9       Computing prediction loss by Eq. (38)
10       $\mathcal{P} \leftarrow \mathcal{P} - \mu \nabla_{\mathcal{P}} L$ 
11 return  $\mathcal{P}$ 

```

G. Anomaly Detection

The MGCL is trained using the segmented loss function L to achieve optimal performance, as shown in Algorithm 1. Additionally, during the threshold selection phase, traditional MTS anomaly detection methods often adopt the Peak Over Threshold (POT) [25]. However, POT suffers from several drawbacks, including strong dependence on data distribution assumptions, high subjectivity in threshold selection, poor scalability to high dimensions, and sensitivity to noise. To address these limitations, we propose a novel dynamic threshold search strategy, as illustrated in Algorithm 2. It enables more adaptive and robust selection of threshold scores. Finally, during the detection phase, anomaly states of test samples are determined based on the dynamically selected thresholds, achieving more accurate anomaly detection, as demonstrated in Algorithm 3.

V. EXPERIMENT

In this section, we conduct extensive experiments on eight real-world datasets to answer the six research questions.

- 1) *RQ1*: How does MGCL perform compared to baselines in terms of benchmark MTS anomaly detection?
- 2) *RQ2*: How does MGCL perform in traffic anomaly detection with a larger number of variables?
- 3) *RQ3*: How do different components of MGCL influence its performance?
- 4) *RQ4*: How do hyperparameter settings affect MGCL?
- 5) *RQ5*: How is the robustness of MGCL?
- 6) *RQ6*: How does the running time and complexity efficiency of MGCL compare to the baselines?
- 7) *RQ7*: How does MGCL perform in real scenarios?

Algorithm 2 Threshold Selection for Anomaly Score

Input: $\mathcal{Y}$: prediction error
 $\mathcal{L}$: buffer length
 β : upper limit of abnormal proportion
 $[\delta, \partial]$: search interval
 s : search step length
Output: $\mathcal{T}$: the best threshold

```

1 begin
2    $\mathcal{T} \leftarrow 0$ ;  $score_{max} \leftarrow -\infty$ ;
3    $\mathcal{Y}_m \leftarrow mean(\mathcal{Y})$ ;  $\mathcal{Y}_s \leftarrow std(\mathcal{Y})$ 
4   for  $i$  from  $\delta$  to  $\partial$  with  $s$  step do
5      $\mathcal{Y}_{cur} \leftarrow \mathcal{Y}_m + i * \mathcal{Y}_s$ 
6      $\mathcal{Y}_{nor} \leftarrow \{\mathcal{Y}' \in \mathcal{Y} \mid \mathcal{Y}' < \mathcal{Y}_{cur}\}$ 
7      $\mathcal{Y}_{abnor} \leftarrow \{j \mid \mathcal{Y}[j] \geq \mathcal{Y}_{cur}\}$ 
8     Expand  $\mathcal{Y}_{abnor}$  based on  $\mathcal{L}$ 
9     if  $\mathcal{Y}_{abnor}$  not empty then
10       $mean_{de} \leftarrow mean(\mathcal{Y}_{nor})/\mathcal{Y}_m$ 
11       $std_{de} \leftarrow std(\mathcal{Y}_{nor})/\mathcal{Y}_s$ 
12       $score \leftarrow mean_{de} + std_{de}$ 
13      if  $score > score_{max}$  and  $\mathcal{Y}_{abnor}/\mathcal{Y} \leq \beta$  then
14         $score_{max} \leftarrow score$ 
15         $\mathcal{T} \leftarrow \mathcal{Y}_{cur}$ 
16 return  $\mathcal{T}$ 

```

Algorithm 3 Anomaly Detection of MGCL

Input: X_{test} : tested MTS series
 $\mathcal{Y}'$: prediction error in testing
 $\mathcal{T}$: the best threshold
Output: $\mathcal{D}$: anomaly detection status

```

1 begin
2   for  $X'$  in  $X_{test}$  do
3     Getting  $\mathcal{T}$  through Algorithm 2
4     if  $\mathcal{Y}' > \mathcal{T}$  then
5        $\mathcal{D} \leftarrow 1$ 
6     else
7        $\mathcal{D} \leftarrow 0$ 
8 return  $\mathcal{D}$ 

```

A. Datasets

The experiments involved eight MTS datasets covering a variety of application scenarios. The Soil Moisture Active Passive (SMAP) and Mars Science Laboratory Rover (MSL) datasets [29], published by NASA, provide data on soil moisture and the Martian environment, respectively. The server machine dataset (SMD) [13], collected by an internet company, includes data on server operating states. The MSDS dataset [30] contains records from distributed systems. However, as system complexity increases, more systems are required to record an expanding number of variables.

To meet the requirements of MTS anomaly detection in real-world complex scenarios, we construct four traffic datasets specifically designed for large-scale MTS anomaly detection. These datasets include PEMS-ACC4, PEMS-ACC7, PEMS-ACC8, and PEMS-Rainy.

TABLE I
SUMMARY OF EXPERIMENTAL DATASETS

Datasets	Variables	Train	Test	Anomalies (%)
SMAP	25	135,183	427,617	13.13
MSL	55	58,317	73,729	10.72
SMD	38	708,420	708,420	4.16
MSDS	10	146,430	146,430	5.37
PEMS-ACC4	307	10,195	3,398	1.23
PEMS-ACC7	228	16,934	5,644	1.26
PEMS-ACC8	170	10,713	3,571	1.40
PEMS-Rainy	308	63,072	21,024	3.10

Among them, PEMS-ACC4, PEMS-ACC7, and PEMS-ACC8 are traffic accident datasets. They are built using traffic data recorded by the California Performance Measurement System (PEMS) [31] in combination with the Countrywide Traffic Accident Datasets (CTADS) [32]. CTADS contains 1.5 million accident reports collected since March 2016, providing detailed information such as accident types, locations, and durations. PEMS-Rainy [33], derived from our previous research, includes traffic condition data associated with heavy rainfall events. The details of all datasets are summarized in Table I.

B. Experiment Settings and Evaluation Metrics

The experiments are conducted on a computer equipped with Python 3.6, PyTorch 1.12, an NVIDIA GeForce RTX 4090 GPU, and an i9-12900K CPU. These datasets are preprocessed using Z-score normalization. During the training phase, an Adam optimizer is utilized, with the initial learning rate, batch size, and epochs set to 0.001, 64, and 200, respectively. To prevent model overfitting, an early stopping strategy is employed. During the testing phase, model performance is evaluated using Precision (P), Recall (R), $F1$ -score ($F1$), and area under the ROC curve (AUC) [23]. Consistent with prior works, following the point adjustment strategy proposed in [13], [35], if any point is detected as abnormal within a time window, the window is marked as abnormal.

C. Baselines

We compare MGCL with 20 baselines. On benchmark datasets, we select classic MTS anomaly detection models.

- 1) *LSTM-NDT* [29]: Is one of the earliest models to use LSTM for MTS anomaly detection.
- 2) *DAGMM* [35]: Combines AE and Gaussian mixture model for anomaly detection.
- 3) *OmniAnomaly* [13]: Uses RNN to capture temporal dependencies and planar normalization flow to model inter-variable dependencies.
- 4) *MSCRED* [16]: Models temporal dependencies from a multiscale perspective.
- 5) *USAD* [36]: Improves anomaly detection performance using AE with adversarial training.
- 6) *GDN* [15]: Is one of the first models to use GNNs, leveraging attention scores for anomaly detection.
- 7) *TranAD* [21]: Builds on a Transformer framework, proficient in capturing inter-variable dependencies.

TABLE II
PERFORMANCE ON BENCHMARK DATASETS INCLUDES SMAP, MSL, SMD, AND MSDS. BOLD: BEST, UNDERLINE: SECOND BEST

Method	SMAP				MSL				SMD				MSDS			
	P	R	F1	AUC	P	R	F1	AUC	P	R	F1	AUC	P	R	F1	AUC
LSTM-NDT	85.22	73.25	78.78	86.01	62.87	99.91	77.18	95.31	97.35	84.39	90.41	96.70	99.89	80.11	88.91	80.12
DAGMM	80.68	98.90	88.87	98.84	73.62	99.97	84.80	97.15	91.02	99.13	94.90	99.53	98.90	80.25	88.60	90.12
OmniAnomaly	81.29	94.18	87.26	98.88	78.47	99.23	87.64	97.81	88.80	<u>99.84</u>	94.00	99.45	99.95	79.63	88.64	89.81
MSCRED	81.75	92.16	86.64	98.21	89.12	98.62	93.63	98.07	72.76	99.74	84.14	99.20	99.94	79.82	88.75	89.42
USAD	74.80	96.27	84.19	98.90	79.49	99.12	88.23	97.95	90.61	99.75	94.96	99.34	99.13	79.60	88.29	89.80
GDN	74.80	98.91	85.19	98.65	93.08	98.92	<u>95.91</u>	98.15	71.71	99.73	83.43	99.25	99.90	80.27	89.02	<u>91.06</u>
TranAD	80.43	<u>99.99</u>	89.15	98.87	90.38	99.99	94.94	99.16	92.62	99.74	96.05	<u>99.74</u>	99.95	81.17	89.59	90.12
CAE-M	81.83	85.67	83.71	99.02	77.51	99.98	87.32	99.04	90.81	96.70	93.66	<u>97.82</u>	99.09	84.39	91.15	90.15
CrossFuN	89.53	91.57	90.54	98.65	92.36	95.98	94.14	98.06	93.15	97.17	95.12	98.11	99.13	80.93	89.11	89.26
DTAAD	82.20	99.99	90.22	99.11	90.38	<u>99.99</u>	94.94	99.18	84.63	99.74	91.47	98.92	99.99	80.26	89.05	90.13
MTGFlow_cluster	<u>92.46</u>	94.55	<u>93.49</u>	<u>99.17</u>	<u>93.48</u>	96.16	94.80	<u>99.33</u>	<u>93.67</u>	98.82	<u>96.18</u>	99.46	98.96	80.02	88.49	89.77
MGCL	94.33	98.92	96.57	99.47	96.65	99.99	98.29	99.46	97.72	99.91	98.80	99.87	<u>99.96</u>	<u>82.24</u>	<u>90.24</u>	91.67

- 8) *CAE-M* [37]: Combines LSTM and AE to capture temporal and inter-variable dependencies, suitable for complex MTS data.
- 9) *CrossFuN* [2]: Considers both time domain and frequency domain information of time series.
- 10) *DTAAD* [23]: Based on Transformer framework, integrates Autoregressive (AR) modeling and AE to achieve strong anomaly detection performance.
- 11) *MTGFlow_Cluster* [38]: Combines a clustering strategy with an entity-aware normalizing flow.

On traffic datasets, we select models specifically designed for traffic tasks.

- 1) *ASTGCN* [39]: Utilizes attention mechanisms to capture spatial-temporal dependencies in traffic states.
- 2) *AGCRN* [40]: Combines GCN and GRU to capture spatial-temporal dependencies.
- 3) *MTGNN* [41]: Enhances spatial-temporal dependencies modeling through Mix-Hop propagation and dilated convolutions.
- 4) *STG-NCDE* [42]: Models traffic patterns using neural controlled differential equations.
- 5) *DDGCRN* [43]: Introduces a dynamic graph convolutional recurrent module for effective spatial-temporal dependencies modeling.
- 6) *MegaCRN* [44]: Proposes a meta-graph learner to capture spatial-temporal dependencies.
- 7) *ST-Wave* [45]: Leverages a wavelet decomposition to disentangle and fuse spatial-temporal dependencies.
- 8) *iTransformer* [46]: Enhances performance by utilizing attention mechanisms on inverted time-series dimensions.
- 9) *DeepSTUQ* [47]: Considers spatial-temporal uncertainties and effectively captures traffic dependencies.

D. Experimental Comparison and Analysis on Benchmark MTS Datasets (RQ1)

To verify the general applicability of MGCL in MTS anomaly detection tasks, we conduct comparative experiments and analyses on four benchmark datasets, including SMAP, MSL, SMD, and MSDS, as shown in Table II. Early models, such as LSTM-NDT and DAGMM, primarily focus on mod-

eling temporal dependencies while neglecting inter-variable dependencies, resulting in poor performance. AE, as an effective mechanism for capturing inter-variable dependencies, is widely adopted due to its ability to efficiently encode MTS. Based on the AE architecture, models like OmniAnomaly, MSCRED, USAD, and CAE-M successfully capture inter-variable dependencies. Leveraging the advantages of GNN in modeling non-Euclidean structures, GDN and CrossFuN effectively model inter-variable dependencies. Moreover, thanks to advancements in attention mechanisms, TranAD, DTAAD, and MTGFlow_cluster achieve remarkable detection performance.

Although the above methods achieve notable performance, they remain inferior to the proposed MGCL. Its advantages are mainly reflected in two aspects.

- 1) MGCL incorporates the multiscale temporal module that comprehensively captures short-term trend dependencies and long-term seasonal dependencies. This design enables better adaptation to the multilevel characteristics of time series.
- 2) MGCL introduces the DMGCN module, integrating a historical pattern bank and dynamic graphs. It dynamically captures normal inter-variable patterns, thereby enhancing anomaly detection performance.

However, MGCL shows suboptimal *F1* performance on the MSDS dataset. This may be attributed to the relatively small number of variables in MSDS, which constrains high-order inter-variable interactions. Consequently, the effectiveness of the adaptive hypergraph generator and 3D-HGCN is limited in such scenarios.

E. Performance Comparison and Analysis on Traffic Datasets (RQ2)

As system complexity increases, the number of variables grows correspondingly. To meet the requirements of real-world detection, we conduct experiments on traffic datasets with a larger number of variables, as shown in Table III. ASTGCN, based on a spatial-temporal GNN framework and incorporating an attention mechanism, effectively captures spatial-temporal dependencies in traffic patterns. AGCRN and MTGNN focus more on the dynamic characteristics of traffic patterns, enhancing their ability to model spatial-temporal

TABLE III
PERFORMANCE ON TRAFFIC DATASETS INCLUDES PEMS-ACC4, PEMS-ACC7, PEMS-ACC8, AND PEMS-RAINY.
BOLD: BEST, UNDERLINE: SECOND BEST

Method	PEMS-ACC4				PEMS-ACC7				PEMS-ACC8				PEMS-Rainy			
	P	R	F1	AUC	P	R	F1	AUC	P	R	F1	AUC	P	R	F1	AUC
ASTGCN	81.63	83.25	82.43	77.26	78.11	79.95	79.02	73.33	82.98	78.37	80.61	74.65	79.20	77.36	78.27	76.91
AGCRN	83.79	84.55	84.17	79.45	79.83	83.97	81.85	74.82	82.93	83.69	83.31	77.02	82.70	80.02	81.34	77.34
MTGNN	86.36	85.37	85.86	81.15	81.89	84.47	83.16	75.63	85.55	86.01	85.78	78.71	82.57	81.46	82.01	78.08
STG-NCDE	87.27	88.42	87.84	79.02	<u>82.40</u>	85.73	84.03	70.62	87.27	85.47	86.36	80.78	82.43	82.93	82.68	73.84
DDGCRN	86.75	88.04	87.39	83.53	80.21	91.12	85.32	83.27	87.59	87.83	87.71	83.74	<u>86.67</u>	84.21	85.42	81.44
MegaCRN	<u>88.85</u>	88.37	88.61	84.87	81.48	90.74	85.86	81.56	86.54	88.11	87.32	84.06	84.51	85.37	84.94	<u>83.17</u>
ST-Wave	84.06	91.28	87.52	<u>85.13</u>	80.51	90.27	85.11	82.83	84.89	<u>91.58</u>	<u>88.11</u>	85.82	86.37	85.20	85.78	82.23
iTransformer	85.82	89.62	87.68	84.45	77.62	<u>92.29</u>	84.32	83.15	95.28	79.54	86.70	78.75	82.45	85.65	84.02	80.97
DeepSTUQ	87.13	<u>91.53</u>	<u>89.28</u>	85.02	80.64	91.93	<u>85.92</u>	<u>83.31</u>	85.39	88.01	86.68	84.28	86.14	<u>86.19</u>	<u>86.16</u>	82.94
MGCL	89.88	92.41	91.13	87.11	83.33	92.66	87.75	84.94	<u>89.18</u>	91.74	90.44	86.74	87.87	89.16	88.51	85.51

dependencies. STG-NCDE introduces differential equations into deep neural networks, modeling continuous spatial-temporal dependencies. DDGCRN, MegaCRN, and ST-Wave demonstrate strong performance by incorporating dynamic graph mechanisms. iTransformer and DeepSTUQ, as the latest methods, exhibit excellent detection performance.

However, none of them outperforms MGCL. This is mainly due to: 1) the proposed adaptive hypergraph generator dynamically constructs hypergraphs, accurately representing high-order inter-variable interactions and 2) the 3D-HGCN module effectively extracts hypergraph features, capturing complex high-order inter-variable dependencies. It is worth noting that, as shown in Table III, compared with other state-of-the-art baselines, MGCL achieves significant performance improvements on PEMS-ACC4 (307 variables), PEMS-ACC7 (228 variables), and PEMS-Rainy (308 variables). This can be attributed to the hypergraph mechanism, which simplifies graph structures by grouping multiple similar variables into a hyperedge, thereby capturing complex inter-variable dependencies. However, its improvement on PEMS-ACC8 (170 variables) is relatively limited. The smaller number of variables in PEMS-ACC8 reduces the effectiveness of hypergraph simplification, constraining MGCL's performance on PEMS-ACC8.

F. Ablation Study (RQ3)

To evaluate the contribution of key components in MGCL, we design the following ablation variants.

- 1) *w/o Trend*: Removes the trend series.
- 2) *w/o Season*: Excludes the seasonal series.
- 3) *w/o Trend-T*: Preserves the trend series, replacing GTCN with a conventional TCN.
- 4) *w/o Season-A*: Retains the seasonal series and replaces TGAN with a conventional attention mechanism.
- 5) *w/o Cross*: Substitutes the cross-learner module with a conventional fusion mechanism.
- 6) *w/o DMGCN*: Replaces DMGCN with a standard GCN.
- 7) *w/o 3D-HGCN*: Applies a traditional hypergraph generation approach and replaces 3D-HGCN with a standard hypergraph convolution.

We conduct an ablation study on six datasets, as shown in Fig. 4. The w/o 3D-HGCN significantly affects the performance of MGCL. This is attributed to the adaptive

hypergraph generator's ability to dynamically construct reasonable hypergraphs, while 3D-HGCN excels in modeling multiorder inter-variable dependencies. The results of w/o DMGCN highlight the essential role of DMGCN, as it not only captures normal patterns through the historical pattern bank of variables but also effectively models low-order inter-variable dependencies. The performance of w/o Cross indicates that the absence of the cross-learner module results in insufficient complementary learning and deep fusion between low-order and high-order dependencies. Furthermore, the w/o Trend-T and w/o Season-A results underscore the critical roles of GTCN and TGAN, respectively. Compared to conventional temporal feature extraction modules, they enhance the modeling of long- and short-term dependencies in time series. Additionally, the results of w/o Trend and w/o Season, respectively, reveal the importance of capturing both trend and seasonal series, both of which contribute to model performance enhancement. All in all, each key component of MGCL plays a positive role, aligning with the design objectives.

G. Parameter Sensitivity Analysis (RQ4)

To further explore the impact of different hyperparameters on MGCL, we analyze three key hyperparameters: time window size, threshold search interval $[\delta, \theta]$ of anomaly score, and diffusion steps of AGCN. The results on six datasets are shown in Fig. 5. Time window size, as a critical parameter, significantly affects the model's performance. A large window size may capture excessive redundant information, increasing the risk of overfitting. Conversely, an overly small window size may fail to capture sufficient temporal dependencies, resulting in the loss of critical information. As observed, when the window size surpasses a certain threshold, the performance generally declines.

Threshold search interval $[\delta, \theta]$ anomaly score directly affects anomaly detection performance. A smaller search interval enables fine-grained threshold adjustment but may fail to encompass a global optimal threshold, preventing the model from achieving peak performance. Conversely, a larger search interval broadens the search scope but may introduce excessive invalid thresholds, increasing computational costs and potentially introducing noise, leading to performance fluctuations.

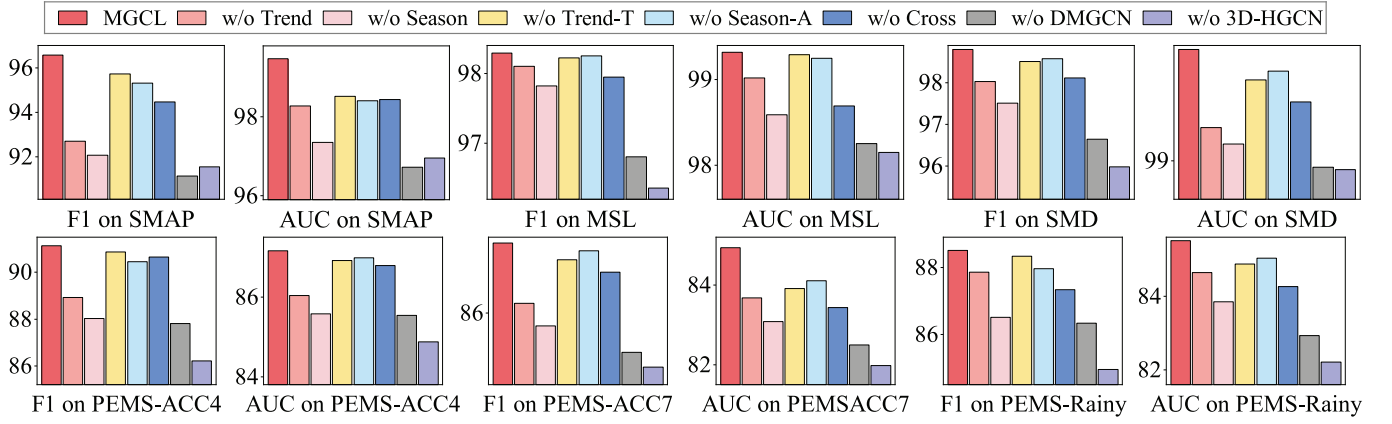


Fig. 4. Ablation study results on SMAP, MSL, SMD, PEMS-ACC4, PEMS-ACC7, and PEMS-Rainy datasets.

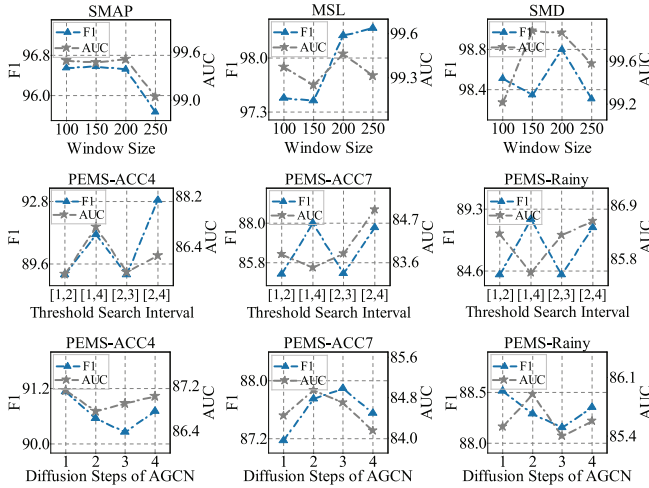


Fig. 5. Hyperparameter study results on SMAP, MSL, SMD, PEMS-ACC4, PEMS-ACC7, and PEMS-Rainy datasets.

Previous studies indicate that increasing the diffusion step of GCN can expand the spatial perception range of the model, enhancing its ability to capture inter-variable dependencies. However, as shown in Fig. 5, performance deteriorates when the diffusion step exceeds a certain threshold. We attribute this to the fact that 3D-HGCN already possesses robust spatial perception capabilities. Excessive diffusion steps may introduce redundant information and noise, leading to inaccuracies in hypergraph generation and ultimately degrading the performance.

H. Robustness Testing (RQ5)

In real-world scenarios, sudden events such as traffic accidents or communication interruptions can cause sensor failures, leading to data loss or errors. Therefore, we conduct two robustness tests to facilitate the application of MGCL in practical scenarios. Specifically, on PEMS-ACC4, PEMS-ACC7, and PEMS-ACC8 datasets, we randomly erase 5% to 20% of the data to simulate data missing. In addition, we add 10% to 50% Gaussian noise (mean 10, standard deviation 60) to simulate data errors. Comparison results with AGCRN,

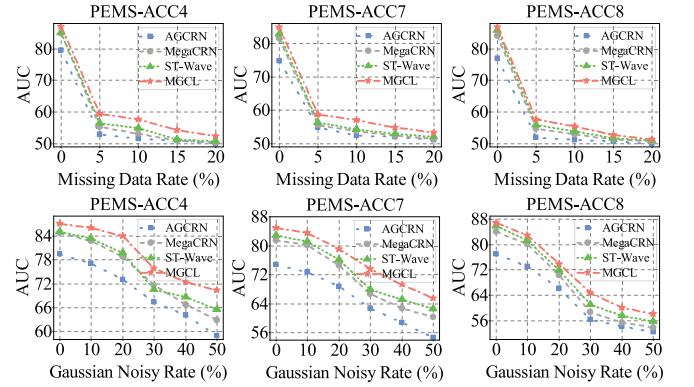


Fig. 6. Robustness testing results on PEMS-ACC4, PEMS-ACC7, and PEMS-ACC8 datasets.

MegaCRN, and ST-Wave are shown in Fig. 6. As data loss rates or noise levels increase, MGCL demonstrates superior robustness with a relatively slower performance degradation trend compared to these baselines. This robustness stems from the combined effects of the multiscale temporal module and the multiorder graph cross-learning module. In the inter-variable dimension, the multiorder graph cross-learning module effectively captures both low-order and high-order inter-variable dependencies. Additionally, it facilitates complementary learning between these dependencies, mitigating the impact of data missing and errors. In the temporal dimension, the multiscale temporal module captures comprehensive temporal dependencies by modeling trends and seasonal series, ensuring a more robust representation of temporal patterns.

I. Complexity Analysis (RQ6)

Model complexity is a key indicator for evaluating efficiency in practical applications. We perform a comprehensive complexity analysis on the PEMS-Rainy dataset, as shown in Fig. 7. AGCRN, a lightweight architecture based on adaptive graphs and RNN, significantly reduces computational overhead. However, its low $F1$ -score reveals limitations in detection performance. MegaCRN enhances detection performance by incorporating a meta-graph learner, while ST-

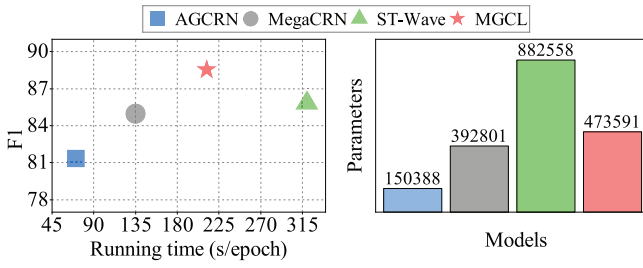


Fig. 7. Running time and parameter count of models on PEMS-Rainy dataset. The running time per epoch encompasses training, validation, and testing time, while the number of parameters reflects the model's complexity.

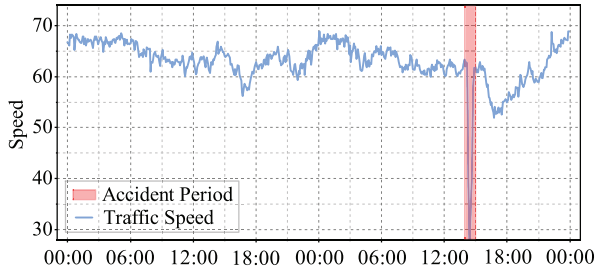


Fig. 8. Traffic speed variation curve at node No.137 near the accident site in California on January 10, 2019 shows significant fluctuations during the accident period (red shaded area).

Wave employs a decoupling fusion framework with graph wavelet position encoding to model inter-variable dependencies. Although ST-Wave improves detection accuracy, it incurs higher runtime costs. In contrast, MGCL effectively balances accuracy and complexity, making it well-suited for real-world applications.

J. Case Study (RQ7)

1) *Visualization of Adaptive Hypergraphs*: To evaluate the effectiveness of the adaptive hypergraph generator, we conduct experiments on the PEMS-Rainy dataset. Specifically, on January 10, 2019, at 14:15, a level-3 traffic accident occurred in downtown California, lasting approximately 2 h. We visualize traffic speed changes at node No.137 near the accident site, as shown in Fig. 8. This curve reveals a significant impact of the accident on traffic patterns. Furthermore, we visualize hypergraphs during the accident period. Fig. 9(a) shows the hypergraph before the accident. As shown in Fig. 9(b), the accident triggered notable structural changes, with a newly formed hyperedge (marked in red) emerging in the affected downtown area. Fig. 9(c) and (d) shows that as the impact of the accident spreads, the number of nodes associated with the red hyperedge grows as congestion propagates. Conversely, as emergency measures such as traffic control are implemented, the accident's impact gradually diminishes. Fig. 9(e) and (f) demonstrates a reduction in the number of nodes covered by the red hyperedge. The above results show that compared with traditional static hypergraphs, the adaptive hypergraph generator can not only generate dynamic hypergraphs but also explicitly represent the diffusion and dissipation process of

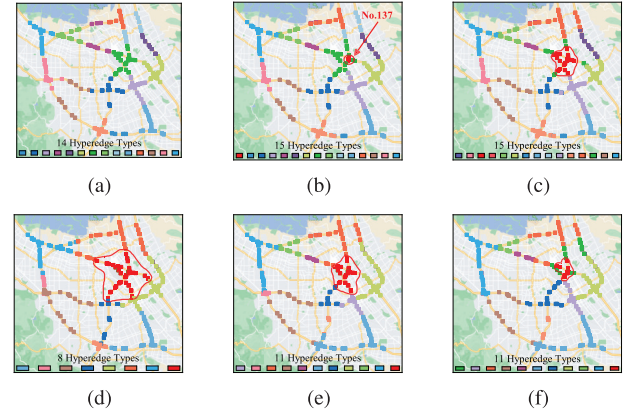


Fig. 9. Visualization of the hypergraph during the accident near node No.137. Nodes in different colors indicate that they belong to different hyperedges. From the occurrence to the propagation to the dissipation of the accident, the dynamic changes in hyperedge types and the spatial-temporal evolution of the circled regions intuitively reveal the propagation and recovery mechanisms of the traffic network under the impact of the accident. (a) At 14:00. (b) At 14:15. (c) At 14:30. (d) At 14:45. (e) At 15:00. (f) At 15:15.

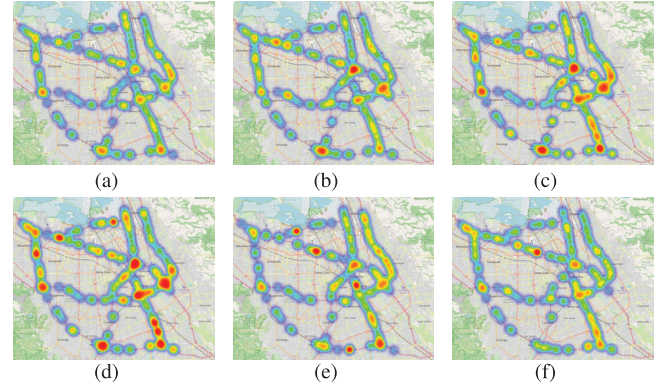


Fig. 10. Visualization of spatial-temporal anomaly scores during the heavy rainfall in California on February 14, 2019. Darker colors indicate higher anomaly scores at the corresponding locations. (a) At 11:00. (b) At 12:30. (c) At 14:00. (d) At 15:30. (e) At 17:00. (f) At 18:30.

anomalous events, providing valuable insights for resource allocation and management.

2) *Anomaly Localization and Diagnosis*: Accurate anomaly localization and diagnosis are key for MTS anomaly detection, enabling rapid identification and resolution of anomalies. To evaluate MGCL's capability in anomaly localization and diagnosis, we conduct experiments on PEMS-Rainy. Specifically, on February 14, 2019, a heavy rainfall event occurred in California. The spatial-temporal anomaly score map of the affected area, as shown in Fig. 10, reveals distinct patterns of anomaly distribution. Overall, during the rainfall period, the intensity and extent of the rainfall typically increase initially and then decrease. Correspondingly, the number of high anomaly score points follows a similar trend, first increasing and then diminishing. However, at 17:00, despite a reduction in rainfall intensity, certain regions still exhibit relatively high anomaly scores. These areas are primarily concentrated in Silicon Valley, downtown, and key public transportation hubs. We hypothesize that this phenomenon is related to the evening rush hour at that time. The rainfall exacerbates traffic congestion in these already busy regions, significantly increasing traffic

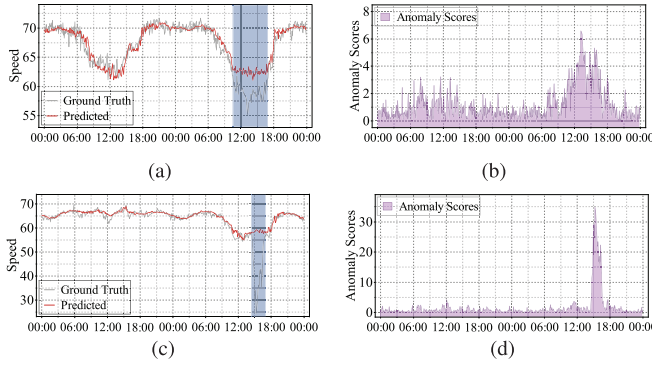


Fig. 11. Visualization of predictions and anomaly scores for nodes No.39 and No.271. (a) Prediction curve for node No.39. (b) Anomaly score curve for node No.39. (c) Prediction curve for node No.271. (d) Anomaly score curve for node No.271.

pressure and consequently leading to sustained high anomaly scores.

Furthermore, we visualize the traffic speed predictions and anomaly scores for nodes No.39 and No.271. These nodes exhibit high connectivity within the adjacency matrix, representing critical locations in the road network. Fig. 11(a) and (c) demonstrates that MGCL accurately predicts normal traffic patterns while effectively amplifying the differences between predictions and ground truth during anomalous periods, thereby maximizing anomaly scores. Fig. 11(b) and (d) presents the time-varying anomaly score curves for nodes No.39 and No.271, revealing that anomaly scores during anomalous periods are significantly higher than those in non-anomalous periods. The case study results indicate that MGCL not only accurately localizes anomalous variables but also effectively detects anomalous periods.

In summary, experimental results from various aspects show that MGCL achieves state-of-the-art anomaly detection performance and can be effectively applied to MTS anomaly detection in the real world.

VI. CONCLUSION

In this article, we propose an MGCL framework for MTS anomaly detection. The multiscale temporal module, built with TGAN and GTCN, effectively captures both long-term seasonal and short-term trend dependencies in time series. Additionally, we design an adaptive hypergraph generator that dynamically constructs hypergraphs to represent high-order inter-variable interactions. Furthermore, by integrating DMGCN, 3D-HGCN, and a cross-learner, MGCL deeply models multiorder inter-variable dependencies. Experimental results on eight real-world datasets demonstrate that MGCL significantly outperforms state-of-the-art baselines. Moreover, we validate the robustness of MGCL under scenarios involving data loss and errors. Through the visualization of adaptive hypergraphs and spatial-temporal anomaly score maps, MGCL exhibits outstanding capabilities in anomaly localization and diagnosis. However, MGCL demands substantial computational resources when processing datasets with a large number of variables. In future work, we aim to explore lightweight

techniques to enhance computational efficiency and practicality.

REFERENCES

- [1] J. Wang, S. Shao, Y. Bai, J. Deng, and Y. Lin, "Multiscale wavelet graph AutoEncoder for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.
- [2] Y. Bai, J. Wang, X. Zhang, X. Miao, and Y. Lin, "CrossFuN: Multiview joint cross-fusion network for time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–9, 2023.
- [3] C. Li, L. Guo, H. Gao, and Y. Li, "Similarity-measured isolation forest: Anomaly detection method for machine monitoring data," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.
- [4] M. Canizo, I. Triguero, A. Conde, and E. Onieva, "Multi-head CNN-RNN for multi-time series anomaly detection: An industrial case study," *Neurocomputing*, vol. 363, pp. 246–260, Oct. 2019.
- [5] A. J. T. Sukanda and D. Adytia, "Wave forecast using bidirectional GRU and GRU method case study in Pangandaran, Indonesia," in *Proc. Int. Conf. Data Sci. Appl. (ICoDSA)*, 2022, pp. 278–282.
- [6] H. Abbasimehr, M. Shabani, and M. Yousefi, "An optimized model using LSTM network for demand forecasting," *Comput. Ind. Eng.*, vol. 143, May 2020, Art. no. 106435.
- [7] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Dec. 2024.
- [8] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Comput. Surv.*, vol. 55, no. 6, p. 109, 2023.
- [9] J. Bi, H. Yuan, S. Li, K. Zhang, J. Zhang, and M. Zhou, "ARIMA-based and multiapplication workload prediction with wavelet decomposition and Savitzky–Golay filter in clouds," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 4, pp. 2495–2506, Apr. 2024.
- [10] Y. Jin, C. Qiu, L. Sun, X. Peng, and J. Zhou, "Anomaly detection in time series via robust PCA," in *Proc. 2nd IEEE Int. Conf. Intell. Transp. Eng. (ICITE)*, Sep. 2017, pp. 352–355.
- [11] C. Huang, G. Min, Y. Wu, Y. Ying, K. Pei, and Z. Xiang, "Time series anomaly detection for trustworthy services in cloud computing systems," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 60–72, Feb. 2022.
- [12] C. Dong, J. Tao, Q. Chao, H. Yu, and C. Liu, "Subsequence time series clustering-based unsupervised approach for anomaly detection of axial piston pumps," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023.
- [13] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Mining*, 2019, pp. 2828–2837.
- [14] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1003–1012.
- [15] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, May 2021, pp. 4027–4035.
- [16] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1409–1416.
- [17] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.
- [18] L. Chen et al., "Multi-scale adaptive graph neural network for multivariate time series forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10748–10761, Oct. 2023.
- [19] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 4419–4425.
- [20] W. Zhang, C. Zhang, and F. Tsung, "GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 2390–2397.
- [21] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, Feb. 2022.
- [22] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. 10th Int. Conf. Learn. Represent.*, 2022, pp. 1–20.
- [23] L. Yu, Q.-H. Lu, and Y. Xue, "DTAAD: Dual tcn-attention networks for anomaly detection in multivariate time series data," *Knowl.-Based Syst.*, vol. 295, Jul. 2024, Art. no. 111849.

- [24] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 3558–3565.
- [25] H. Pang et al., "Asymptotic consistent graph structure learning for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–10, 2024.
- [26] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "HyperGCN: A new method for training graph convolutional networks on hypergraphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 1509–1520.
- [27] J. Yi and J. Park, "Hypergraph convolutional recurrent neural network," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 3366–3376.
- [28] H. Poostchi and M. Piccardi, "BiLSTM-SSVM: Training the BiLSTM with a structured Hinge loss for named-entity recognition," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 203–212, Feb. 2022.
- [29] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 387–395.
- [30] S. Nedelkoski, J. Bogatinovski, A. K. Mandapati, S. Becker, J. Cardoso, and O. Kao, "Multi-source distributed system data for AI-powered analytics," in *Proc. 8th IFIP WG 2.14 Eur. Conf. (ESOCC)*, Crete, Greece. Cham, Switzerland: Springer, 2020, pp. 161–176.
- [31] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 914–921.
- [32] S. Moosavi, M. H. Samavatian, S. Parthasarathy, R. Teodorescu, and R. Ramnath, "Accident risk prediction based on heterogeneous sparse data: New dataset and insights," in *Proc. 27th ACM SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2019, pp. 33–42.
- [33] Z. Cui et al., "Predictive and multigranularity resilience assessment of urban transportation based on neural controlled differential equation," *IEEE Trans. Reliab.*, early access, Dec. 27, 2024, doi: [10.1109/TR.2024.3514712](https://doi.org/10.1109/TR.2024.3514712).
- [34] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [35] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. ICLR*, Feb. 2018, pp. 1–12.
- [36] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2020, pp. 3395–3404.
- [37] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, "Unsupervised deep anomaly detection for multi-sensor time-series signals," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
- [38] Q. Zhou, S. He, H. Liu, J. Chen, and W. Meng, "Label-free multivariate time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3166–3179, Jul. 2024.
- [39] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. 33rd AAAI Conf. Artif. Intell. (AAAI), 31st Innov. Appl. Artif. Intell. Conf.*, 2019, pp. 922–929.
- [40] L. Bai, L. Yao, C. Li, and X. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Proc. NIPS*, 2020, pp. 17804–17815.
- [41] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 753–763.
- [42] J. Choi, H. Choi, J. Hwang, and N. Park, "Graph neural controlled differential equations for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 6, pp. 6367–6374.
- [43] W. Weng et al., "A decomposition dynamic graph convolutional recurrent network for traffic forecasting," *Pattern Recognit.*, vol. 142, Oct. 2023, Art. no. 109670.
- [44] R. Jiang et al., "Spatio-temporal meta-graph learning for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 7, 2023, pp. 8078–8086.
- [45] Y. Fang et al., "When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks," in *Proc. IEEE 39th Int. Conf. Data Eng. (ICDE)*, Apr. 2023, pp. 517–529.
- [46] Y. Liu et al., "ITransformer: Inverted transformers are effective for time series forecasting," in *Proc. ICLR*, Jan. 2023, pp. 1–12.
- [47] W. Qian, Y. Zhao, D. Zhang, B. Chen, K. Zheng, and X. Zhou, "Towards a unified understanding of uncertainty quantification in traffic flow forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 5, pp. 2239–2256, May 2024.



Zhe Cui was born in Shandong, China, in 1998. He received the B.S. degree from Shandong University of Science and Technology, Qingdao, China, in 2019. He is currently pursuing the Ph.D. degree with Tongji University, Shanghai, China.

His current research interests include deep learning, intelligent transportation systems, and data mining.



Di Zang received the Ph.D. degree from Kiel University, Kiel, Germany, in 2007.

She was a Post-Doctoral Researcher with the University of Minnesota, Minneapolis, MN, USA, in 2008. She is currently an Associate Professor with Tongji University, Shanghai, China. Her current research interests include deep learning, intelligent transportation systems, and data mining.



Junqi Zhang (Senior Member, IEEE) received the Ph.D. degree in computing science from Fudan University, Shanghai, China, in 2007.

He was a Post-Doctoral Research Fellow with Peking University, Beijing, China, in 2007. From 2014 to 2015, he was a Visiting Scholar with New Jersey Institute of Technology, Newark, NJ, USA. He is currently a Full Professor with Tongji University, Shanghai. He has published more than 20 articles in IEEE TRANSACTIONS and more than 60 papers in conferences. His current research interests include swarm intelligence, swarm robots, multiagent systems, learning automata, reinforcement learning, and big data.

Dr. Zhang was a recipient of the Outstanding Postdoctoral Award.



Keshuang Tang received the Ph.D. degree in traffic engineering from Nagoya University, Nagoya, Japan, in 2008.

He has worked as a Post-Doctoral Research Fellow with The University of Tokyo, Tokyo, Japan, and as a Project Assistant Professor with the University of Tohoku, Sendai, Japan. He was a Visiting Scholar with the University of California at Berkeley, Berkeley, CA, USA, in 2010. He is currently a Full Professor with the College of Transportation Engineering, Tongji University, Shanghai, China. His

main research interests include driver behavior, signal control, and intelligent transportation systems.